ЛР №11 — дослідженська частина (§4). ShowFinder і TVmaze API

Тема: AI-агент для інтеграції REST API: Fetch API, Promise та async/await, аналіз API-документації, loading та error states, debugging через Network.

Репозиторій	https://github.com/HunterDead14/Web_React (тека `Laba_11/`)
Опублікована адреса	https://hunterdead14.github.io/Web_React/Laba_11/ · самоперевірка: …`/Laba_11/check.html`
Конспект API	*`Laba_11/API.md`* *(./API.md)* — §4.5
Правила для агента	`Laba_11/AGENTS.md` — буде створено після аудиту C1–C2 (ТЗ, §4)
Дата роботи	19.10.2026
AI-агент	OpenCode (Kodeo/OpenCode CLI), режим агента з редагуванням файлів
Модель	`minimax-m3.1-flash-free` (провайдер `opencode`)
Термін здачі	22.10.2026

> ***Як отримано результати.*** *Усі 13 експериментів прогнано в* ***справжньому Chrome*** *на сторінці* `Laba_11/experiments.html`*, піднятій локальним сервером (*`http://127.0.0.1:8123/experiments.html`*).*
> *Console і панель Network знімались програмно з того самого браузера, тому в колонці «Фактичний результат» — дослівний вивід, а не переказ. Кожен експеримент виконувався на* ***чистій сторінці*** *(новий таб), щоб стан кешу й Console не перетікав між тестами.*
>
> ⚠️ ***Про колонку «Мій аналіз».*** *Це чернетка аналізу, зібрана з нотаток під час підготовки. §5.2 лабораторної вимагає: «Мій аналіз пишете самі, без AI-агента й до запуску». Перед здачею перепиши цю колонку* ***своїми словами*** *— щоб це був твій аналіз, а не конспект. Фактичні результати й розбіжності вже перевірені — їх не треба змінювати.*

1. Таблиця експериментів (§6, п. 1 звіту)

#	Що досліджено	Мій аналіз	Фактичний результат (Console і Network)	Пояснення (якщо не збіглося)

#	Що досліджено	Мій аналіз	Фактичний результат (Console і Network)	Пояснення (якщо не збіглося)

1	**fetch і статус 404.** `GET /shows/999999999`	`fetch` не знає про HTTP-статуси як про помилки: він знає лише «мережа працює / не працює». Статус — просто число в `Response`. Тому проміс не відхилиться, а `catch` не спрацює. Очікував `ok=false, status=404` і JSON-тіло помилки TVmaze.	**Console:**
			`false 404`
			`{name: 'Not Found', message: '', code: 0, status: 404}`
			червоний рядок: `Failed to load resource: the server responded with a status of 404 ()`
			Network: 1 рядок, `GET https://api.tvmaze.com/shows/999999999` → **404**, 276 мс
				Розбіжність: я прогнозував «у Console нічого червоного» — **є червоний рядок**. Але він від браузера (мережевий лог), а не від нашого коду: наш код відпрацював штатно. Черговий рядок не означає, що «код упав» — саме це й питає лабораторна.

2	**Тіло відповіді читається один раз** + HTML замість JSON	Тіло — одноразовий потік байтів; після першого читання він «спустошений». Друге читання — помилка стану об'єкта, а не HTTP-помилка. HTML з JSON-парсером дає іншу помилку: `SyntaxError` на першому символі.	**Console:**
			`Game of Thrones`
			`вдруге: TypeError | Failed to execute 'json' on 'Response': body stream already read`
			`HTML content-type: text/html | status 200`
			`HTML: SyntaxError Unexpected token '<', "<!doctype "... is not valid JSON`
			Network: `GET /shows/82` → 200; `GET experiments.html` → 200, `content-type: text/html`
				Розбіжності:
				1) `id=82` — це **Game of Thrones**, а не «The Cosby Show», як я записав у чернетці.
				2) Текст помилки у вашому Chrome: `body stream already read` (Chrome 152), а не «Body is unusable» — це лише формулювання.
				3) Різниця `TypeError` (прочитав двічі) проти `SyntaxError` (сервер дав не JSON) — підтвердилася.

3	**`forEach`, `for…of` і `Promise.all`**	`forEach` не чекає на проміси: він синхронно проходить масив, тому `counts.length` = 0 одразу. `for…of` з `await` — послідовно, `Promise.all` — паралельно, тому друга дорівняє сумі затримок, а третя — максимальній.	**Console:**
			`forEach: 0`
			`через 2 с: 3`
			`for…of: 208 мс`
			`Promise.all: 155 мс`
			Network (Waterfall), 9 запитів:
			• `forEach`: старт `0/1/1 мс` → **паралельно**
			• `for…of`: старт `2003 → 2093 → 2143 мс` → **поспіль**
			• `Promise.all`: старт `2212/2212/2212 мс` → **паралельно**
				Розбіжність: я прогнозував ≈**3×** різницю, фактично **208 проти 155 мс ≈ 1,3×**. Причина — мала затримка й warm-з'єднання: кожен запит у послідовному варіанті уже «на гарячому» каналі. **Структурно** Waterfall доводить головне: поспіль проти паралельно.

4	**`return` і `return await` у `try`**	`try/catch` ловить синхронні винятки. `return fetch(url)` — це повернення проміса: `try` завершується **до** відхилення, тому помилка «проходить мимо» локального `catch` і дістається до зовнішнього `.catch`.	**Console:** `поза load: TypeError` (варіант `return fetch(url)`) `catch у load: TypeError` (варіант `return await fetch(url)`) `варіант із return await -> null` **Network:** 2 запити до `https://api.tvmaze.invalid/shows/82` → `failed`, `net::ERR_TUNNEL_CONNECTION_FAILED`	Підтвердилося. **Уточнення для звіту:** у нашому середовищі браузер іде через проксі-тунель, тому помилка — `ERR_TUNNEL_CONNECTION_FAILED`; у звичайному браузері без проксі буде `net::ERR_NAME_NOT_RESOLVED` (домен `.invalid` не існує). Для JS-коду це одна й та сама помилка: `TypeError` / `Failed to fetch`.

#	Що досліджено	Мій аналіз	Фактичний результат (Console і Network)	Пояснення (якщо не збіглося)

5	**`Promise.all` і `Promise.allSettled`**	`Promise.all` відхиляється на першій помилці й **відкидає** решту — дві вдалі відповіді, які вже прийшли, втрачено. `allSettled` не відхиляється ніколи: кожен результат має `{ status, value | reason }`.	**Console:**
			`all: TypeError | Failed to fetch`
			`allSettled: ["fulfilled","rejected","fulfilled"]`
			`значення fulfilled: 200,200`
			`причина rejected: TypeError`
			Network: у фазі `all` бачимо `GET /shows/82` → **200** і `GET /shows/139` → **200** (обидва вдали!), але результату немає — проміс уже відхилився.
				Підтвердилося без застережень. `allSettled` дав рівно `["fulfilled","rejected","fulfilled"]`.

6	**`AbortController`**	Скасування до відповіді → `AbortError`. Скасування **після** отримання заголовків не обнуляє вже отриманий `Response` — `status` і `ok` лишаються дійсними, але тіло вже не прочитається.	**Console:**
			`до відповіді: AbortError | signal is aborted without reason`
			`після відповіді: 200 | ok: true | type: cors`
			`json(): AbortError | The user aborted a request.`
			Network: `GET /search/shows?q=girls` → `net::ERR_ABORTED`; `GET /shows/139` → **200**, але `failed / net::ERR_ABORTED`
				Підтвердилося. **Уточнення:** другий запит устиг отримати статус 200 (заголовки прийшли), але читання тіла обірвалося — саме це і показує `json(): AbortError`. У панелі Network такий запит має статус **`(canceled)`**.

7	**Заголовки відповіді**	Браузер показує скрипту лише CORS-safelisted заголовки: `Cache-Control`, `Content-Language`, `Content-Length`, `Content-Type`, `Expires`, `Last-Modified`, `Pragma`. `Date`, `Server`, `Retry-After` приховані, поки сервер не додасть `Access-Control-Expose-Headers`.	**Console:**
			`content-type: application/json; charset=UTF-8`
			`cache-control: public, max-age=3600`
			`date: null`
			`server: null`
			`retry-after: null`
			`ключі: Array(2) → ['cache-control', 'content-type']`
			Network → Response Headers (DevTools бачить усе):
			`access-control-allow-origin: *`
			`cache-control: public, max-age=3600`
			`content-encoding: gzip`
			`content-type: application/json; charset=UTF-8`
			`date: Mon, 19 Oct 2026 20:54:38 GMT`
			`server: nginx/1.24.0 (Ubuntu)`
			`vary: Accept-Encoding`
				Підтвердилося. **Уточнення:** скрипт бачить **рівно 2** заголовки, а не 3 — `content-length` відсутній, бо TVmaze віддає `transfer-encoding: chunked` (і `content-encoding: gzip`). `date` і `server` сервер надсилає, але до скрипта не доходять.

#	Що досліджено	Мій аналіз	Фактичний результат (Console і Network)	Пояснення (якщо не збіглося)

8	**HTTP-кеш**	`Cache-Control: public, max-age=3600` → браузер зберігає відповідь на годину; другий запит не йде в мережу. `cache: 'no-store'` = не читати кеш **і** не зберігати.	**Console (час виконання):**
			`1 звичайний: 159 мс, статус 200`
			`2 звичайний: 6 мс, статус 200`
			`3 no-store: 57 мс, статус 200`
			`4 звичайний (повтор): 6 мс, статус 200`
			Network: та сама картина — 1-й запит 156 мс, 2-й 4 мс, 3-й 54 мс, 4-й 4 мс
				Підтвердилося. **Уточнення (цікаве):** 4-й запит знову з кешу — `no-store` **не перезаписує** вже збережену відповідь, а лише не читає і не зберігає її. Тому після `no-store` звичайний запит знову миттєвий.

9	**Переадресація**	`fetch` сам іде за переадресацією, тому клієнт ніколи не бачить 301. Після переадресації `status=200`, `redirected=true`, а `url` — фінальна адреса.	**Console:**
			`200 true https://api.tvmaze.com/shows/82`
			`Game of Thrones`
			Network — два рядки:
			`GET https://api.tvmaze.com/lookup/shows?imdb=tt0944947` → **301** (64 мс)
			`GET https://api.tvmaze.com/shows/82` → **200** (3 мс)
				Підтвердилося. **Уточнення:** IMDb-id `tt0944947` — це **Game of Thrones**, TVmaze-id **82** (той самий, що в експерименті 2).

10	**Рядок запиту**	`&` у рядку запиту — роздільник параметрів, тож `Law & Order` без кодування розривається на `q=Law` і окремий параметр `Order`. `URLSearchParams` кодує значення сам.	**Console:**
			`Law  Array(2)` → `q = "Law "`, ключів **2**
			`Law & Order ?q=Law+%26+Order`
			`сирий рядок plain: https://api.tvmaze.com/search/shows?q=Law%20&%20Order`
			`сирий рядок encoded: https://api.tvmaze.com/search/shows?q=Law+%26+Order`
				Підтвердилося без застережень. Перша адреса має **два** параметри (`q` і порожній `Order`), друга — один, але правильно закодований.

11	**Два домени TVmaze**	`api.tvmaze.com` і `www.tvmaze.com` — різні origin (протокол + домен + порт), хоча домен один. Запит на інший origin без `Access-Control-Allow-Origin` блокується **браузером** — і блокується саме *читання* відповіді, а не надсилання.	**Console:**
			`api: 200`
			червоний рядок: `Access to fetch at 'https://www.tvmaze.com/shows/82' from origin 'http://127.0.0.1:8123' has been blocked by CORS policy: No 'Access-Control-Allow-Origin' header is present on the requested resource.`
			`www: TypeError Failed to fetch`
			Network: рядок `GET https://www.tvmaze.com/shows/82` є, `failed / net::ERR_FAILED` (236 мс)
				Уточнення до формулювання: запит **дійшов** до сервера, і відповідь **прийшла** — інакше браузер не зміг би написати «No 'Access-Control-Allow-Origin' header is present»: цю помилку він може дати лише після отримання відповіді. Заблоковано доступ скрипта до її тіла. У панелі Network рядок позначено `failed / net::ERR_FAILED`.

#	Що досліджено	Мій аналіз	Фактичний результат (Console і Network)	Пояснення (якщо не збіглося)

12	**Preflight**	«Простий» запит — це GET/HEAD/POST + лише safelisted-заголовки. `Content-Type: application/json` — **не** safelisted, тому браузер спершу надсилає `OPTIONS` (preflight) і чекає дозволу. `Accept` — safelisted, preflight не потрібен.	**Console:**
			червоний рядок: `Access to fetch at 'https://api.tvmaze.com/shows/82' ... Response to preflight request doesn't pass access control check: No 'Access-Control-Allow-Origin' header is present ...`
			`Content-Type: TypeError Failed to fetch`
			`Accept: 200`
			Network — три рядки:
			`GET /shows/82` → `failed / net::ERR_FAILED`
			`OPTIONS /shows/82` → **405** (98 мс)
			`GET /shows/82` → **200** (3 мс, це вже запит з `Accept`)
				Підтвердилося. **Уточнення:** на перший запит припадає **два** рядки — `OPTIONS 405` і провалений `GET`; сам `GET` з `Content-Type` **не дійшов** до сервера. Другий запит (з `Accept`) — один рядок, без `OPTIONS`. Тобто «All endpoints are CORS enabled» означає дозвіл **простих** запитів, а не будь-яких заголовків.

13	**`mode: 'no-cors'`**	`no-cors` не «виправляє» CORS, а посилює обмеження: відповідь стає **opaque** — браузер приховує від скрипта все (статус, заголовки, тіло). Помилка зникає разом із даними.	**Console:**
			`opaque 0 false`
			`""`
			Network: `GET https://www.tvmaze.com/shows/82` → **301**, далі `GET https://www.tvmaze.com/shows/82/game-of-thrones` → `failed / net::ERR_ABORTED`
				Підтвердилося: `type='opaque'`, `status=0`, `ok=false`, тіло — **порожній рядок `""`** (не помилка!). **Цікавий дрібниць:** сайт TVmaze віддає `301` на канонічну адресу `/shows/82/game-of-thrones` — і цю переадресацію браузер простежує і в `no-cors`-режимі, але скрипту нічого про неї не каже.

2. Деталі по кожному експерименту

§4.1 Fetch API і Promise

Експеримент 1 · fetch і статус 404

```
const response = await fetch('https://api.tvmaze.com/shows/999999999');
console.log(response.ok, response.status);
console.log(await response.json());
```

```
[Console] false 404
[Console] {name: 'Not Found', message: '', code: 0, status: 404}
[Console] Failed to load resource: the server responded with a status of 404 ()   ← рядок від браузера
[Network] GET https://api.tvmaze.com/shows/999999999 → 404, 276 мс
```

Висновок. `fetch` не відхиляє проміс на 404: HTTP-статус — це **дані**, а не помилка JS. `catch` не спрацює. Після `await fetch(...)` обов'язково перевіряти `response.ok`, інакше 404/500 прийдуть
в застосунок як дані (ТЗ, С3.1). Червоний рядок у Console належить браузеру і не означає, що наш код упав.

Де це трапиться: пошук і сторінка серіалу — відповідь 404 чи 500, яку ніхто не перевірив через `response.ok`, потрапляє в застосунок як дані.

Експеримент 2 · Тіло відповіді читається один раз

```
const response = await fetch('https://api.tvmaze.com/shows/82');
console.log((await response.json()).name);
try { await response.json(); } catch (error) { console.log('вдруге:', error.name); }
const page = await fetch('experiments.html');
try { await page.json(); } catch (error) { console.log('HTML:', error.name, error.message); }
```

```
[Console] Game of Thrones
[Console] вдруге: TypeError | Failed to execute 'json' on 'Response': body stream already read
[Console] HTML content-type: text/html | status 200
[Console] HTML: SyntaxError Unexpected token '<', "<!doctype "... is not valid JSON
[Network] GET https://api.tvmaze.com/shows/82 → 200
[Network] GET http://127.0.0.1:8123/experiments.html → 200, text/html
```

Висновок. Два різні діагнози — і їх треба розрізняти:

Помилка	Що означає
`TypeError: body stream already read`	ми самі прочитали тіло **двічі** — помилка стану об'єкта
`SyntaxError: Unexpected token '<'`	сервер дав **не JSON** (проксі, captive portal, сторінка помилки зі статусом 200)

Де це трапиться: код, що читає тіло «для логування», ламає справжнє читання.

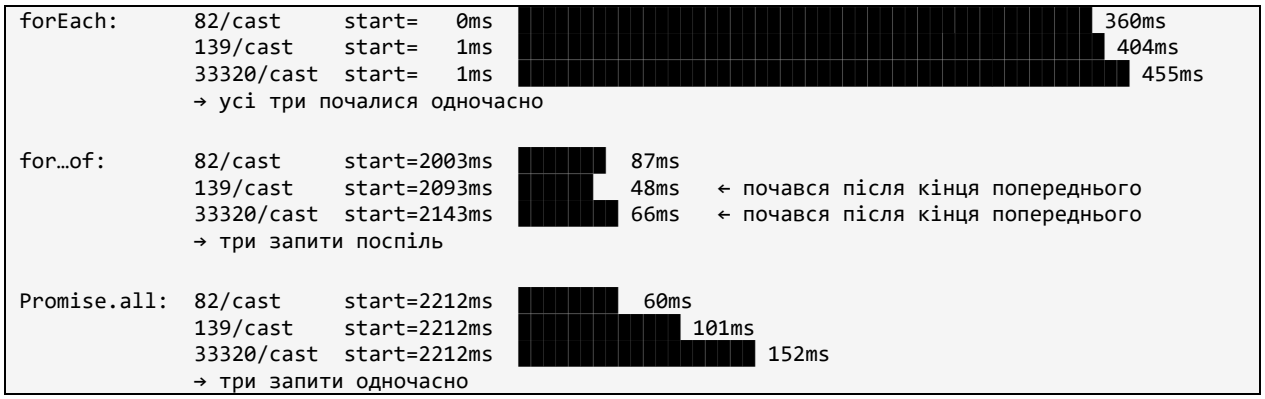
Експеримент 3 · forEach, for…of і Promise.all

```
const ids = [82, 139, 33320];
const load = async (id) => (await fetch(`https://api.tvmaze.com/shows/${id}/cast`, { cache: 'no-store' })).json();
const counts = [];
ids.forEach(async (id) => counts.push((await load(id)).length));
console.log('forEach:', counts.length);
await new Promise((resolve) => setTimeout(resolve, 2000));
console.log('через 2 с:', counts.length);
let start = performance.now();
for (const id of ids) await load(id);
```

```
console.log('for…of:', Math.round(performance.now() - start), 'мс');
start = performance.now();
await Promise.all(ids.map(load));
console.log('Promise.all:', Math.round(performance.now() - start), 'мс');
```

```
[Console] forEach: 0
[Console] через 2 с: 3
[Console] for…of: 208 мс
[Console] Promise.all: 155 мс
```

Waterfall (start від першого запиту, три групи по три):



Висновок. `forEach` не чекає на проміси (`counts.length` = 0 одразу, через 2 с — 3).

Послідовний варіант — це сума затримок, паралельний — максимум із трьох. **Числовий коефіцієнт

залежить від затримки** (у нас 208 проти 155 мс, ≈1,3×, бо з'єднання було вже warm), але

структура Waterfall доводить головне: поспіль проти паралельно.

Де це трапиться: сторінка серіалу — епізоди й актори, запитані один за одним, це кілька

затримок поспіль; а з `forEach` стан «завантаження» закінчується **раніше**, ніж приходять дані.

Експеримент 4 · return і return await у try

```
async function load(url) {
  try { return fetch(url); } catch (error) { console.log('catch у load:', error.name); return
null; }
}
const result = await load('https://api.tvmaze.invalid/shows/82').catch((error) => `поза load:
${error.name}`);
console.log(result);
```

```
[Console] поза load: TypeError            ← варіант «return fetch(url)»
[Console] catch у load: TypeError         ← варіант «return await fetch(url)»
[Console] варіант із return await -> null
[Network] 2 запити до https://api.tvmaze.invalid/shows/82 → failed
(net::ERR_TUNNEL_CONNECTION_FAILED)
```

Висновок. `try/catch` навколо `return fetch(…)` **нічого не ловить**: `try` завершується до відхилення проміса. З `return await fetch(…)` помилка виникає всередині `try` і ловиться. `Uncaught (in promise)` — це те саме, але вже на рівні консолі браузера.

Експеримент 5 · Promise.all і Promise.allSettled

```
const urls = ['https://api.tvmaze.com/shows/82', 'https://api.tvmaze.invalid/shows/1',
'https://api.tvmaze.com/shows/139'];
try {
  const responses = await Promise.all(urls.map((url) => fetch(url)));
  console.log('all:', responses.map((response) => response.status));
} catch (error) { console.log('all:', error.name); }
```

```
const settled = await Promise.allSettled(urls.map((url) => fetch(url)));
console.log('allSettled:', settled.map((result) => result.status));
```

```
[Console] all: TypeError | Failed to fetch
[Console] allSettled: ["fulfilled","rejected","fulfilled"]
[Console] значення fulfilled: 200,200
[Console] причина rejected: TypeError
[Network] у фазі all: GET /shows/82 → 200, GET /shows/139 → 200   ← обидва вдали, але результату
немає
```

Висновок. `Promise.all` — «все або нічого»: перша помилка відкидає решту, і дві вдалі відповіді, які вже прийшли, **втрачено** (їхні тіла навіть не прочитані).
`Promise.allSettled` не відхиляється ніколи й дає результат по кожному запиту окремо.

Експеримент 6 · AbortController

```
const early = new AbortController();
const request = fetch('https://api.tvmaze.com/search/shows?q=girls', { signal: early.signal });
early.abort();
try { await request; } catch (error) { console.log('до відповіді:', error.name); }
const late = new AbortController();
const response = await fetch('https://api.tvmaze.com/shows/139', { signal: late.signal });
late.abort();
console.log('після відповіді:', response.status);
try { await response.json(); } catch (error) { console.log('json():', error.name); }
```

```
[Console] до відповіді: AbortError | signal is aborted without reason
[Console] після відповіді: 200 | ok: true | type: cors
[Console] json(): AbortError | The user aborted a request.
[Network] GET /search/shows?q=girls → net::ERR_ABORTED          (canceled)
[Network] GET /shows/139 → 200, failed / net::ERR_ABORTED        (canceled)
```

Висновок. `AbortError` — **не помилка**. Скасування після отримання заголовків не обнуляє вже отриманий `Response`: `status` і `ok` лишаються дійсними, але тіло вже не прочитається.
У панелі Network скасовані запити мають статус **`(canceled)`**.

§4.2 HTTP: статуси, заголовки, кеш

Експеримент 7 · Заголовки відповіді

```
const response = await fetch('https://api.tvmaze.com/shows/82');
console.log(response.headers.get('content-type'));
console.log(response.headers.get('cache-control'));
console.log(response.headers.get('date'));
console.log([...response.headers.keys()]);
```

```
[Console] content-type: application/json; charset=UTF-8
[Console] cache-control: public, max-age=3600
[Console] date: null
[Console] server: null
[Console] retry-after: null
[Console] ключі: Array(2)   → ['cache-control', 'content-type']

[Network → Response Headers — те, що бачить DevTools]
  access-control-allow-origin: *
  cache-control: public, max-age=3600
  content-encoding: gzip
  content-type: application/json; charset=UTF-8
  date: Mon, 19 Oct 2026 20:54:38 GMT
  server: nginx/1.24.0 (Ubuntu)
  vary: Accept-Encoding
```

Висновок. Скрипт бачить лише 2 заголовки. `date` і `server` сервер **надсилає**, але вони не

CORS-safelisted — і без `Access-Control-Expose-Headers` до скрипта не доходять. Тому `Retry-After`, який документація радить читати на 429, ми теж не побачимо: **паузу 2–5 с задаємо самі** (ТЗ, С3.6).

Експеримент 8 · HTTP-кеш

```
await fetch('https://api.tvmaze.com/shows/1');
await fetch('https://api.tvmaze.com/shows/1');
await fetch('https://api.tvmaze.com/shows/1', { cache: 'no-store' });
```

```
[Console] 1 звичайний: 159 мс, статус 200
[Console] 2 звичайний: 6 мс, статус 200          ← з кешу
[Console] 3 no-store:  57 мс, статус 200         ← у мережу
[Console] 4 звичайний (повтор): 6 мс, статус 200 ← знову з кешу
[Network] тривалість: 156 мс → 4 мс → 54 мс → 4 мс
```

Висновок. `max-age=3600` керує кешем **браузера**: другий запит обслуговується з кешу за кілька мілісекунд. `cache: 'no-store'` вимикає кеш для цього запиту, але **не перезаписує** вже збережену відповідь — тому 4-й запит знову миттєвий. Повторюючи експеримент, перший запит
теж прийде з кешу: відповідь живе 3600 с.

Де це трапиться: повторне відкриття серіалу. Навіщо застосунку власний кеш, якщо браузер уже кешує відповіді? Бо браузер кешує **HTTP-відповідь**, а застосунок мусить не робити запит зовсім (ТЗ, С2.4) — це власний `Map` у пам'яті, і тільки для успішних відповідей (С3.8).

Експеримент 9 · Переадресація

```
const response = await fetch('https://api.tvmaze.com/lookup/shows?imdb=tt0944947');
console.log(response.status, response.redirected, response.url);
console.log((await response.json()).name);
```

```
[Console] 200 true https://api.tvmaze.com/shows/82
[Console] Game of Thrones
[Network] GET https://api.tvmaze.com/lookup/shows?imdb=tt0944947 → 301, 64 мс
[Network] GET https://api.tvmaze.com/shows/82                     → 200,  3 мс
```

Висновок. `fetch` сам іде за переадресацією: клієнт бачить `200`, `redirected=true` і фінальну `url`. Статус `301` бачить **браузер** (окремий рядок у Network), а не скрипт.
Тому в розборі статусів 301 у застосунку бути не може.

Експеримент 10 · Рядок запиту

```
const query = 'Law & Order';
const plain = new URL(`https://api.tvmaze.com/search/shows?q=${query}`);
const encoded = new URL(`https://api.tvmaze.com/search/shows?${new URLSearchParams({ q: query
})}`);
console.log(plain.searchParams.get('q'), [...plain.searchParams.keys()]);
console.log(encoded.searchParams.get('q'), encoded.search);
```

```
[Console] Law  Array(2)          → q = "Law " (із пробілом), параметрів 2
[Console] Law & Order ?q=Law+%26+Order
[Console] сирий рядок plain:   https://api.tvmaze.com/search/shows?q=Law%20&%20Order
[Console] сирий рядок encoded: https://api.tvmaze.com/search/shows?q=Law+%26+Order
```

Висновок. Без кодування `&` розриває значення: API отримує «Law» замість «Law & Order». `URLSearchParams` кодує сам (`&` → `%26`, пробіл → `+`, а сервер декодує `+` назад у пробіл).

Де це трапиться: пошук (ТЗ, С1.1) — запит «Law & Order», вставлений у шаблонний рядок без

кодування, шукає «Law».

§4.3 CORS

Експеримент 11 · Два домени TVmaze

```
const api = await fetch('https://api.tvmaze.com/shows/82');
console.log('api:', api.status);
try {
  const site = await fetch('https://www.tvmaze.com/shows/82');
  console.log('www:', site.status);
} catch (error) { console.log('www:', error.name, error.message); }
```

```
[Console] api: 200
[Console] Access to fetch at 'https://www.tvmaze.com/shows/82' from origin 'http://127.0.0.1:8123'
          has been blocked by CORS policy: No 'Access-Control-Allow-Origin' header is present
          on the requested resource.                          ← червоний рядок браузера
[Console] www: TypeError Failed to fetch
[Network] GET https://www.tvmaze.com/shows/82 → failed / net::ERR_FAILED, 236 мс
```

Висновок. Запит **дійшов** до сервера і відповідь **прийшла** — інакше браузер не зміг би написати «No 'Access-Control-Allow-Origin' header is present»: таку помилку він може дати лише після отримання відповіді. Заблоковано **читання** відповіді, а не надсилання запиту.
`origin` = протокол + домен + порт; `api.tvmaze.com` і `www.tvmaze.com` — різні origin.

Експеримент 12 · Preflight

```
try {
  const response = await fetch('https://api.tvmaze.com/shows/82', { headers: { 'Content-Type':
'application/json' } });
  console.log('Content-Type:', response.status);
} catch (error) { console.log('Content-Type:', error.name, error.message); }
const response = await fetch('https://api.tvmaze.com/shows/82', { headers: { Accept:
'application/json' } });
console.log('Accept:', response.status);
```

```
[Console] Access to fetch at 'https://api.tvmaze.com/shows/82' from origin 'http://127.0.0.1:8123'
          has been blocked by CORS policy: Response to preflight request doesn't pass access
          control check: No 'Access-Control-Allow-Origin' header is present on the requested
resource.
[Console] Content-Type: TypeError Failed to fetch
[Console] Accept: 200
[Network] GET    /shows/82 → failed / net::ERR_FAILED   ← сам GET не дійшов
[Network] OPTIONS /shows/82 → 405, 98 мс               ← preflight відхилено
[Network] GET    /shows/82 → 200, 3 мс                 ← це вже запит із заголовком Accept
```

Висновок. `Content-Type: application/json` **не** є CORS-safelisted (дозволені лише `text/plain`, `application/x-www-form-urlencoded`, `multipart/form-data`), тому браузер спершу
надсилає `OPTIONS`. TVmaze відповідає на `OPTIONS` кодом **405** без CORS-заголовків → запит падає. `Accept` — safelisted, preflight не потрібен.
Документація TVmaze каже: «All endpoints are CORS enabled». Це означає, що дозволені **прості** запити; preflight — інший механізм, і TVmaze на `OPTIONS` не відповідає.

Експеримент 13 · mode: 'no-cors'

```
const response = await fetch('https://www.tvmaze.com/shows/82', { mode: 'no-cors' });
console.log(response.type, response.status, response.ok);
console.log(JSON.stringify(await response.text()));
```

```
[Console] opaque 0 false
[Console] ""
```

```
[Network] GET https://www.tvmaze.com/shows/82                          → 301
[Network] GET https://www.tvmaze.com/shows/82/game-of-thrones          → failed / net::ERR_ABORTED
```

Висновок. `no-cors` не «виправляє» CORS — він посилює обмеження. Відповідь стає **opaque**: `type='opaque'`, `status=0`, `ok=false`, тіло — порожній рядок `""` (без помилки). Помилка зникає, але разом із нею зникають і дані. Тому це непридатне рішення.

3. §4.4 Панель Network

Окремої таблиці для цього розділу немає — панель Network знадобилася в кожному експерименті вище.

Що з неї взято:

Що дивилися	Де	Результат
колонка **Status**	екс. 1, 6, 9, 12	`404`, `(canceled)` / `net::ERR_ABORTED`, `301`, `OPTIONS 405`
колонка **Size**	екс. 8	кешована відповідь обслуговується за 4–6 мс замість 156 мс
Waterfall	екс. 3	`forEach` і `Promise.all` — паралельно; `for…of` — поспіль
вкладка **Headers → Response**	екс. 7	`date`, `server` є в відповіді, але не видні скрипту
Initiator	екс. 1–13	рядок коду, що надіслав запит
Console → рівні помилок	екс. 1, 4, 11, 12	червоний рядок від браузера ≠ помилка нашого коду
фільтр **Fetch/XHR**	усі	прибирає шум від завантаження сторінки
Preserve log	екс. 9, 12	бачимо обидва рядки переадресації / preflight

4. §4.5 Документація API → API.md

Конспект — окремим файлом: ***`Laba_11/API.md`*** *(./API.md)*. Він містить таблицю «Питання → Що каже документація → Що показав запит (адреса)» за всіма десятьма пунктами завдання.

Таблиця заголовків відповіді, форм відповідей і статусів — нижче, у розділах 2 і 4 цього файлу.

Кожен рядок API.md перевірено справжнім `GET`-запитом до `https://api.tvmaze.com` (19.10.2026).

Найважливіші розбіжності з документацією, знайдені під час перевірки:

#	Знахідка	Адреса запиту
1	Параметр `page` у `/search/shows` **не підтримується** — `?q=girls&page=1` повертає ті самі 10 id	`/search/shows?q=girls`
2	`embed[]=episodes&embed[]=cast` дає **все одразу в одному запиті**: `_embedded` з 62 епізодами й 8 акторами	`/shows/139?embed[]=episodes&embed[]=cast`
3	Пошук без результатів — це `200` + `[]`, **не** 404 (а `singlesearch` дає 404)	`/search/shows?q=zzqx`

#	Знахідка	Адреса запиту
4	`404` повертає JSON-тіло `{"name":"Not Found",…}`; проміс при цьому не відхиляється	`/shows/999999999`
5	Сервер **не надсилає** жодних лімітних заголовків — паузу 2–5 с задає клієнт	будь-який
6	`network` і `webChannel` — взаємовиключні: у `/shows/254` `network: null`, але `webChannel: "PokerGO"`	`/shows/254`
7	`rating.average` і `premiered` теж можуть бути `null`	`/shows/410`
8	`image` може бути `null`	`/shows/1124`
9	`genres` може бути порожнім масивом `[]`	`/shows/272`
10	Реальні заголовки: `access-control-allow-origin: *`, `cache-control: public, max-age=3600`, `date`, `server: nginx/1.24.0`	`/search/shows?q=girls`

Адреси для ручної перевірки в панелі Network

Вставити в адресний рядок (Network → Fetch/XHR) або в Console — так можна перевірити кожен рядок `API.md` власними очима:

```
https://api.tvmaze.com/search/shows?q=girls
https://api.tvmaze.com/search/shows?q=zzqx
https://api.tvmaze.com/search/shows?q=Law%20%26%20Order
https://api.tvmaze.com/shows/139
https://api.tvmaze.com/shows/139?embed[]=episodes&embed[]=cast
https://api.tvmaze.com/shows/139/episodes
https://api.tvmaze.com/shows/139/cast
https://api.tvmaze.com/shows/254
https://api.tvmaze.com/shows/410
https://api.tvmaze.com/shows/1124
https://api.tvmaze.com/shows/272
https://api.tvmaze.com/shows/999999999
```

> *Ліміт TVmaze — 20 запитів / 10 с з однієї IP-адреси. Не запускайте їх у циклі: на занятті всі виходять з однієї адреси, і можна отримати `429`.*

5. §4.6 Асинхронний код, написаний AI

§4.6 не має окремої таблиці — це **чеклист** помилок, за якими шукаємо в AI-чернетках у §5, і сировина для `AGENTS.md`. Зіставка «помилка → доказ із §4 → правило ТЗ → рядок самоперевірки»:

#	Помилка AI	Доказ	Порушує	Рядок `check.html`
1	Не перевіряє `response.ok` → 404/500 стають «даними»	екс. 1	С3.1, С3.3	`C3.1`, `C3.3`

#	Помилка AI	Доказ	Порушує	Рядок `check.html`
2	Додає `Content-Type: application/json` до GET	екс. 12	C1.9	`C1.9`
3	Запит на кожне натискання клавіші; debounce без скасування застарілих	екс. 3, 6	C1.3, C1.4, C1.8	`C1.3`, `C1.8`
4	Запит у шаблонному рядку без кодування	екс. 10	C1.1	`C1.1`
5	Серіал → епізоди → актори поспіль, без `embed`	екс. 3 + API.md	C2.2	`C2.2`
6	Лишає стан завантаження після помилки (немає `finally`)	—	C3.7	`C3.7`
7	Лише `console.error` або `alert()`	—	C3.10	`C3.10`
8	Показує скасований запит (`AbortError`) як помилку	екс. 6	C3.9, C1.8, C2.5	`C3.9`
9	На `429` повторює запит одразу й без кінця	API.md	C3.6	`C3.6`
10	Читає заголовки, яких браузер скрипту не віддає	екс. 7	C3.6	`C3.6`
11	Вигадує ендпоїнти й параметри, яких немає в документації	API.md	C2.2	`C2.2`
12	«Виправляє» CORS через `mode: 'no-cors'` або проксі	екс. 11, 13	C1.9	`C1.9`

6. Що далі (практична частина, §5–§6)

Крок	Стан
Коміт starter-проєкту без змін (§5.1)	☐
Коміт `API notes` з `API.md` (§5.1)	☐
Challenge 1 → чернетка → аудит → `Challenge 1: fixes`	☐
Challenge 2 → чернетка → аудит → `Challenge 2: fixes`	☐
Коміт `Agent rules` з `AGENTS.md` (≤ 40 рядків)	☐
Challenge 3 → чернетка → аудит → `Challenge 3: fixes`	☐
`check.html` — «Пройдено: 24 з 24»	☐
Чек-лист Network N1–N6 зі скриншотами (ТЗ, §6)	☐
Публікація на GitHub Pages / Vercel	☐
Лист зі звітом (до 22.10.2026)	☐